CashPilot — SME Finance Copilot: Technical Documentation

FinTechathon 2026 International Track · Topic E: SME Finance Copilot
Theme: AI as a Financial Participant
Team: Sampad Chakma (@SamJU25) · Purno Dewan (@coderx86) · Binoy Kumar Chakma (@binoy-kumar-changmha) · Parban Chakma (@Parban1527)
Repository: https://github.com/SamJU25/F-SME-Finance-Copilot
Document version: 1.0 (2026-10-20) · Verified against the submitted `main` branch and a fresh `pytest -q` run (963 passed, 1 skipped)

Contents

1. Problem and scope
2. System architecture
3. Core algorithms
4. AI agent design
5. Security design
6. Data contracts and API
7. Deployment and operations
8. Testing and verification
9. Current limits and roadmap

---

1. Problem and Scope

Small businesses rarely fail because they are unprofitable; they fail because cash arrives later than it leaves. A business that buys stock on day 0, pays its supplier on day 30, sells on day 45 and collects on day 105 carries a 75-day funding gap (Cash Conversion Cycle = DIO 45 + DSO 60 − DPO 30). Owners usually see that gap only when payroll bounces, and bank working-capital underwriting takes weeks.

CashPilot is a finance copilot for the owner of that business. Topic E asks for three capabilities, and CashPilot delivers each one from an uploaded bank statement:

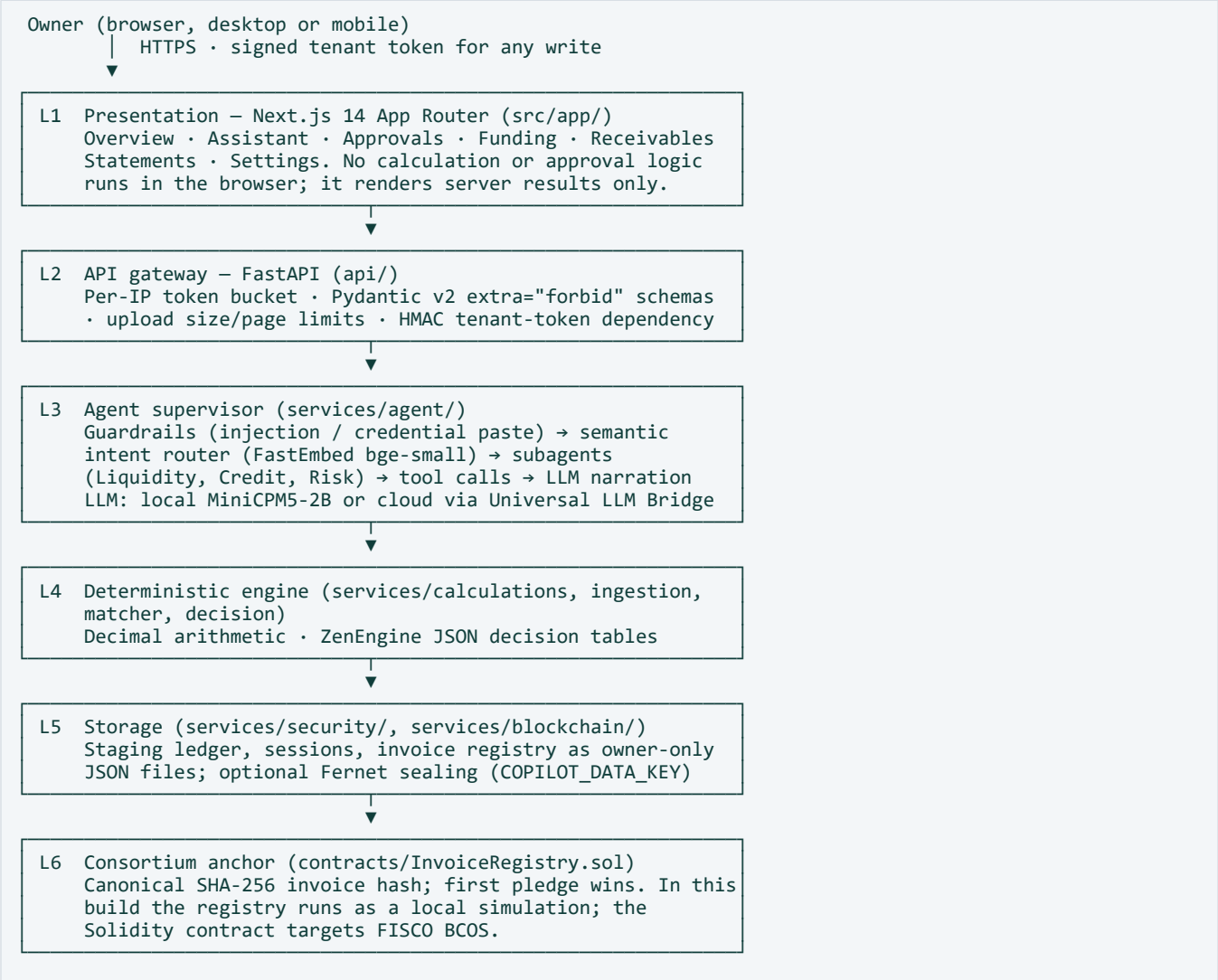
Topic E capability	What CashPilot does	Where it lives
Cash-flow management with alerts	Parses CSV/PDF statements, computes balance, burn, runway, daily average balance, and projects the date the balance goes negative; returns a `cash_status` (`CASH_NONPOSITIVE`, `LOW_RUNWAY`, `WATCH`, `NO_BURN`) against an owner-set alert threshold	`services/ingestion/`, `services/calculations/`, `POST /api/v1/analyze`
Financing-option matching	Scores liquidity 0–100 from transaction behaviour (no collateral or audited accounts needed), matches facilities, converts factor rates to true APR and flags anything above 36% APR as predatory	`services/matcher/`, `services/calculations/bridge_plan.py`, `POST /api/v1/underwrite`
Business financial analysis	25+ deterministic routines: CCC (DSO/DIO/DPO), DSCR, FCCR, Altman Z′, DuPont ROE, ROIC/EVA, DCF/WACC, unit economics, receivables ageing, client concentration, lease-vs-buy	`services/calculations/`

Design stance — the AI participates, but never holds the money. The agent reads, calculates, explains and drafts. It can stage a financing proposal for the owner to review. It has no bank connection, no lender API and no payment credential, so no prompt, bug or exploit can move funds. Irreversible actions end as a pre-filled request the owner sends personally.

---

2. System Architecture

2.1 Topology



2.2 The separation that matters

The architecture splits **language** from **arithmetic** and **proposal** from **execution**:

- The LLM never computes a number. Every figure in a reply comes from a Decimal routine and is injected into the prompt as verified context. The model writes the explanation around it.
- The LLM never acts. Its only side effect is a Tier 2 proposal in the staging ledger, and only when the caller holds a valid tenant token.
- No component anywhere can execute a Tier 3 action. There is no `/execute` route, no payment SDK and no stored bank credential.

2.3 Repository map

Path	Role
`api/main.py`, `api/routes/`	FastAPI app, rate limiter, routes: analyze, underwrite, auth, actions, agent, decision, health
`services/ingestion/`	CSV and PDF statement parsing, column profiling, invoice CSV parsing, multi-account consolidation
`services/calculations/`	25 Decimal financial modules (runway, DAB, predictive cash, CCC, DSCR, Z′, DuPont, DCF, APR, FX…)
`services/matcher/liquidity_matcher.py`	Facility matching and gray-zone triage
`services/decision/`	Decision-engine registry with ZenEngine, rule-fallback and model adapters
`contracts/decision_tables/`	15 JSON policy tables (underwriting, covenants, security gate, usury, persona…)
`services/agent/`	Supervisor, semantic router, subagents, LLM bridge, local SLM engine, session store, URL guard
`services/security/`	Three-tier gate, staging ledger, tenant tokens, encryption, private-file helpers
`services/blockchain/invoice_hasher.py`	Canonical invoice hash and pledge registry
`contracts/InvoiceRegistry.sol`	Solidity 0.8.20 pledge registry contract
`src/app/`	Next.js 14 dashboard (7 workspaces)
`tests/`	964 tests: unit, integration, security, e2e

---

3. Core Algorithms

All money is `decimal.Decimal` with `ROUND_HALF_UP`; JSON responses carry money as strings (`"7500.00"`), never floats. Staged amounts are integer cents.

3.1 Statement ingestion and tie-out

1. Profile columns (date, amount, currency, description, running balance) from CSV headers or PDF line items.
2. Validate each row; rejected rows keep their row number and a reason code: `INVALID_ROW_SHAPE`, `INVALID_DATE`, `INVALID_AMOUNT`, `INVALID_CURRENCY`, `DUPLICATE_TRANSACTION_ID`, `OUT_OF_ORDER_DATE`.
3. Tie out: `opening_balance + Σ credits − Σ debits = closing_balance`, within the tolerance in `reconciliation_tolerance_policy.json`.
4. Eliminate contra transfers between the owner's own accounts so internal moves are not counted as revenue or spend.

A bad row never silently changes a metric: the analysis runs on accepted rows and returns the rejected ones beside it.

3.2 Burn and runway

```
net_period_change = Σ credits − Σ debits
average_daily_burn = max(0, −net_period_change) / period_days
runway_days       = floor(current_balance / average_daily_burn)   if burn > 0
                  = null (not burning)                            otherwise
cash_status       = CASH_NONPOSITIVE (balance ≤ 0) / LOW_RUNWAY (below alert_threshold_days)
                  / WATCH (burning, above threshold) / NO_BURN
```

3.3 Daily Average Balance (DAB)

Over 30/60/90-day windows the engine carries the last closing balance forward across days with no postings (weekends, holidays), then averages: `DAB = (1/N) · Σ closing_balance_day`. Lenders using transaction-based underwriting (the WeBank Weiyidai model) treat DAB as a proxy for repayment capacity in place of audited accounts.

3.4 Predictive cash crunch

`services/calculations/predictive_cash.py` simulates the daily closing balance forward over 30/60/90 days using detected recurring obligations (`detect_obligations`) and historical inflow cadence. It returns the first date the projected balance crosses zero, the shortfall amount, and a severity code mapped by `deficit_severity_policy.json`.

3.5 Revenue volatility

Inflows are bucketed into 7-day windows; `CV = σ / μ`. Tiers come from `revenue_volatility_policy.json`. Volatility feeds both the liquidity score and the cash forecast's confidence.

3.6 Continuous liquidity score (0–100)

From `services/calculations/liquidity_score.py`:

Component	Max points	Rule
Daily Average Balance	50	≥ $10k: 40–50 (scaled above prime); $3k–$10k: 20–40 linear; < $3k: 0–20 proportional
Revenue volatility (CV)	25	CV ≤ 0.30: 25; 0.30–0.80: 25→10; > 0.80: 5
NSF (bounced payment) record	15	0 NSF: +15; 1: +5; 2: −5; ≥ 3: −20
Transaction density	10	≥ 15 tx: 10; ≥ 5: 7; < 5: 4

Score is clamped to 0–100. Tiers: **PRIME ≥ 75**, **NEAR_PRIME 55–74**, **HIGH_RISK < 55**. Applicants near a boundary get a conditional proposal (for example, asset-backed invoice factoring instead of an unsecured line) instead of a flat decline.

3.7 Financing match and Truth-in-Lending APR

For each candidate facility the matcher checks tier eligibility and limit (a multiple of DAB from `underwriting_policy.json`), then converts cost to a comparable APR:

```
APR (factor products) = ((factor_rate − 1) / term_days) × 365 × 100
APR (trade credit, e.g. 2/10 net 30) = (d / (100 − d)) × (365 / (credit_days − discount_days)) × 100
```

Facilities above **36% APR** are tagged predatory (`USURY_COMPLIANCE` decision domain) and excluded from default recommendations. Results are ranked by fit to the need, with true APR and the dollar cost of the requested amount shown for each.

3.8 Business analysis routines

Routine	Formula / output
Cash Conversion Cycle	`CCC = DSO + DIO − DPO`
DSCR / FCCR	`NOI / debt service`; `(EBIT + fixed charges) / (fixed charges + interest)`
Altman Z′ (private firms)	`0.717X1 + 0.847X2 + 3.107X3 + 0.420X4 + 0.998X5`; safe > 2.90, distress < 1.23
DuPont ROE	margin × asset turnover × equity multiplier
ROIC / EVA	`NOPAT / invested capital`; `NOPAT − WACC × invested capital`
DCF / WACC	FCFF discounted at WACC with perpetuity terminal value
Unit economics	ARR, CAC payback, LTV/CAC, NRR, magic number
Receivables ageing / concentration	Ageing buckets, oldest unpaid invoice, top-client share
Currency	ISO 4217 registry (160+ currencies) with per-currency precision

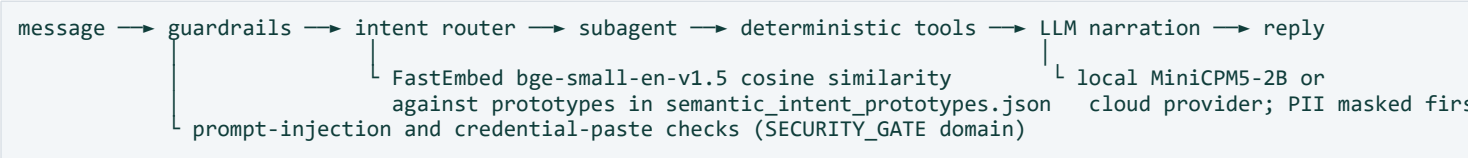
3.9 Policy as data

Thresholds live in 15 JSON decision tables under `contracts/decision_tables/`, evaluated by `DecisionEngineRegistry` (ZenEngine adapter, with a deterministic rule fallback if the engine is unavailable). Changing a covenant or APR ceiling is a reviewed data change, not a code change. Hot reload is operator-only (`DECISION_ADMIN_KEY`).

---

4. AI Agent Design

4.1 Request pipeline (`POST /api/v1/agent/chat`)



1. **Guardrails first.** `inspect_for_prompt_injection` and `inspect_for_api_key` run before anything else. A hit returns `SECURITY_DEFENSE_TRIGGERED` or `CREDENTIAL_SECURITY_ALERT` and stops the pipeline, even when the caller is authenticated and supplied staging parameters (shown in the execution evidence, step 19).
2. **Intent routing.** Embedding similarity selects `CASH_DIAGNOSTIC`, `CREDIT_UNDERWRITING`, `ACTION_STAGING`, `GENERAL_ASSISTANCE` and related intents; when an LLM provider is configured it classifies first, and the embedding router decides otherwise.
3. **Subagents.** `LiquiditySubagent`, `CreditSubagent` and `RiskManagerSubagent` call the Decimal tools and return structured metrics.
4. **Narration.** The selected LLM receives the verified metrics and writes the explanation. A post-processor removes boilerplate, and `numbers_are_grounded()` (`services/agent/prompts.py`) rejects any model reply that quotes a figure absent from the engine output or prompt context; the verified deterministic reply is used instead.
5. **Staging.** If the intent is `ACTION_STAGING` and the caller holds a valid tenant token, the agent writes a Tier 2 proposal and the reply states that nothing has been sent.

4.2 Models

Mode	Model	Notes
Local, air-gapped (default)	`openbmb/MiniCPM5-2B` via PyTorch (`LocalTransformersEngine`)	Thinking mode off by default; weights under `models/base/`; optional deps in `requirements-local.txt`
Cloud	Gemini, Anthropic Claude, OpenAI, DeepSeek, Groq, Ollama, custom OpenAI-compatible	Selected per session; live model discovery via `fetch-models`; endpoints pass the SSRF guard
Fallback	Deterministic summary	Used when no model is available, so the product still answers with correct figures

4.3 Memory

Short-term memory is the last few chat turns plus the session's uploaded statement (`services/agent/session_store.py`). There is no autonomous long-term vector memory: it invites memory poisoning and would let unverified text drift into numeric answers. Durable truth is the parsed statement itself.

---

5. Security Design

The full self-assessment, with the known-risk register, is in `submission/05-security-self-assessment/`. This section summarises the design.

5.1 Three-tier permission model

Tier	What it covers	Who acts	Enforcement
Tier 1 — Observe	Statement analysis, ratios, forecasts, underwriting, chat answers	Agent, autonomously	Read-only routes; no state change beyond the caller's own session
Tier 2 — Draft	Credit-line draws, invoice factoring, transfers as proposals	Agent drafts; owner reviews	`POST /api/v1/actions/stage` requires a signed tenant token; status `STAGED_PENDING_HITL`; 72 h expiry; 5-min dedupe
Tier 3 — Owner-sent	Anything that moves money or commits the business	**Owner only**	No execution route exists. `prepare` returns a pre-filled request the owner sends from their own email or lender portal; `reject` dismisses it. A decided intent cannot change again (409).

Which action types are Tier 3 is itself a decision table (`security_gate_policy.json`), evaluated by `requires_owner_action()`.

5.2 Identity and tenant isolation

- Tenant tokens are HMAC-signed (`TENANT_TOKEN_SECRET`) with expiry; issuance requires an enrollment key bound to a tenant (`TENANT_ENROLLMENT_KEYS` map, or a single key plus `TENANT_ENROLLMENT_TENANT_ID`). An unbound single key is fail-closed.
- Action routes ignore client-supplied `tenant_id` / `X-Tenant-ID` and use the token's tenant. Looking up another tenant's intent returns 404, so IDs cannot be probed.
- Operator routes (decision-table reload) need `DECISION_ADMIN_KEY` and return 503 when it is unset.

5.3 Input and model boundaries

- Pydantic v2 models with `extra="forbid"`: smuggled fields such as `execute_now` return 422; amounts must be positive integers of cents.
- Upload size and PDF page limits before parsing.
- PII masking (IBANs, phone numbers, account numbers, emails) before any text reaches a cloud LLM.
- SSRF guard on user-supplied LLM base URLs: blocks non-HTTP schemes, embedded credentials, loopback, private ranges and cloud metadata hosts.

5.4 Data at rest

Session, ledger and registry files are written owner-only (`0700`/`0600` on POSIX) with atomic replace. Setting `COPILOT_DATA_KEY` seals them with Fernet (`enc:v1:` prefix); `COPILOT_ENV=production|staging` refuses to start without it.

5.5 Invoice double-financing defense

`hash = SHA-256(tax_id ‖ buyer_id ‖ invoice_number ‖ amount_cents)`. The first pledge records the lender; a second pledge of the same invoice to any lender raises `ERR_INVOICE_ALREADY_PLEDGED`. `InvoiceRegistry.sol` implements the same rule for a FISCO BCOS consortium chain, where all participating lenders share one registry.

---

6. Data Contracts and API

All monetary JSON values are decimal strings; staged amounts are integer cents.

Method	Path	Tier / auth	Purpose
GET	`/api/v1/health`	Public	Health and active invariants
POST	`/api/v1/analyze`	Tier 1	Statement upload → cash metrics, rejected rows, forecast
POST	`/api/v1/analyze/invoices`	Tier 1	Receivables ageing from invoice CSV
POST	`/api/v1/analyze/scenario`, `/adjust`, `/summary`	Tier 1	What-if and manual cash adjustments
POST	`/api/v1/underwrite`	Tier 1	Underwriting dossier and matched facilities
POST	`/api/v1/auth/token`	Enrollment key	Issue signed tenant token
POST	`/api/v1/actions/stage`	Tier 2, token	Stage a proposal
GET	`/api/v1/actions`, `/api/v1/actions/{id}`	Token	List / read own proposals
POST	`/api/v1/actions/{id}/prepare`	Tier 3 owner, token	Mark request prepared for the owner to send
POST	`/api/v1/actions/{id}/reject`	Tier 3 owner, token	Dismiss proposal
POST	`/api/v1/agent/chat`	Tier 1; staging needs token	AI CFO dialogue
POST	`/api/v1/agent/test-connection`, `/fetch-models`, `/download-model`, `/delete-model`	Token	Model/provider management
POST	`/api/v1/decision/evaluate`	Tier 1	Evaluate a decision table
POST	`/api/v1/decision/reload`	Operator key	Hot-reload decision tables

Interactive OpenAPI docs: `http://localhost:8000/docs`.

Staged intent schema:

```
{
  "id": "intent-5fbb96e0ceab",
  "tenant_id": "SME-COFFEE-001",
  "action_type": "DRAW_CREDIT_LINE",
  "amount_cents": 2500000,
  "target_identifier": "Receivables Factoring Partner",
  "status": "STAGED_PENDING_HITL",
  "staged_at": "2026-10-20T13:20:21.894188Z"
}
```

---

7. Deployment and Operations

Docker (recommended):

```
git clone https://github.com/SamJU25/F-SME-Finance-Copilot.git
cd F-SME-Finance-Copilot
cp .env.example .env        # set TENANT_ENROLLMENT_KEY + TENANT_ENROLLMENT_TENANT_ID, TENANT_TOKEN_SECRET
docker compose up --build -d
# Dashboard http://localhost:3000 · API http://localhost:8000/api/v1/health · Docs http://localhost:8000/docs
```

Local: Python 3.11+, Node 18+.

```
python -m pip install -r requirements.txt
python -m uvicorn api.main:app --host 127.0.0.1 --port 8000
npm ci && npm run dev -- --hostname 127.0.0.1 --port 3000
```

Optional local model: `pip install -r requirements-local.txt` and `python scripts/download_model.py`.

CI (`.github/workflows/ci.yml`) runs the test suite, builds both container images and smoke-tests the stack: health check, token issuance, anonymous-staging refusal, authenticated staging and sealed ledger.

---

8. Testing and Verification

Suite	Tests	Covers
`tests/unit/`	803	Ingestion, Decimal math, underwriting, decision tables, session store
`tests/integration/`	104	API endpoints, LLM bridge, model discovery, action ledger, session memory
`tests/security/`	55 (1 POSIX-only skip on Windows)	Tier invariants, tenant auth, PII masking, tampering, hardening, encryption, rate limit, SSRF guard
`tests/e2e/`	2	Statement upload → underwriting → staging → owner prepare, via REST and via chat

Latest run (2026-10-20, Windows 11, Python 3.14): **963 passed, 1 skipped in 27.28 s**. Log: `submission/06-execution-evidence/logs/pytest_full_run.log`.

Sandbox evidence run: `python submission/06-execution-evidence/run_sandbox_evidence.py` replays the full operator journey (analysis, underwriting, chat, staging, cross-tenant probes, injection attempts, owner decisions, double pledge) against the real app and writes timestamped logs.

Demo video: `submission/03-demo-video/CashPilot_Demo.mp4` (2:07) records the same flows in the dashboard against the live backend and the on-device model.

---

9. Current Limits and Roadmap

These are stated plainly so reviewers can separate what runs today from what is designed:

Area	Today	Next
Storage	JSON files with process-local locks; single-process only	PostgreSQL 16 with row-level security and append-only ledger tables
Blockchain	Local simulated registry (same hash and first-pledge rule); `.sol` contract not deployed	Deploy `InvoiceRegistry.sol` to a FISCO BCOS test chain with lender nodes
Bank data	Uploaded statements	Read-only open-banking feeds (still no write scope)
Alerts	Status in API and dashboard	Push/email alerts on forecast threshold breach
Auth	HMAC tenant tokens, enrollment keys	OIDC sign-in and per-user roles inside a tenant

The full risk register is in the security self-assessment report.